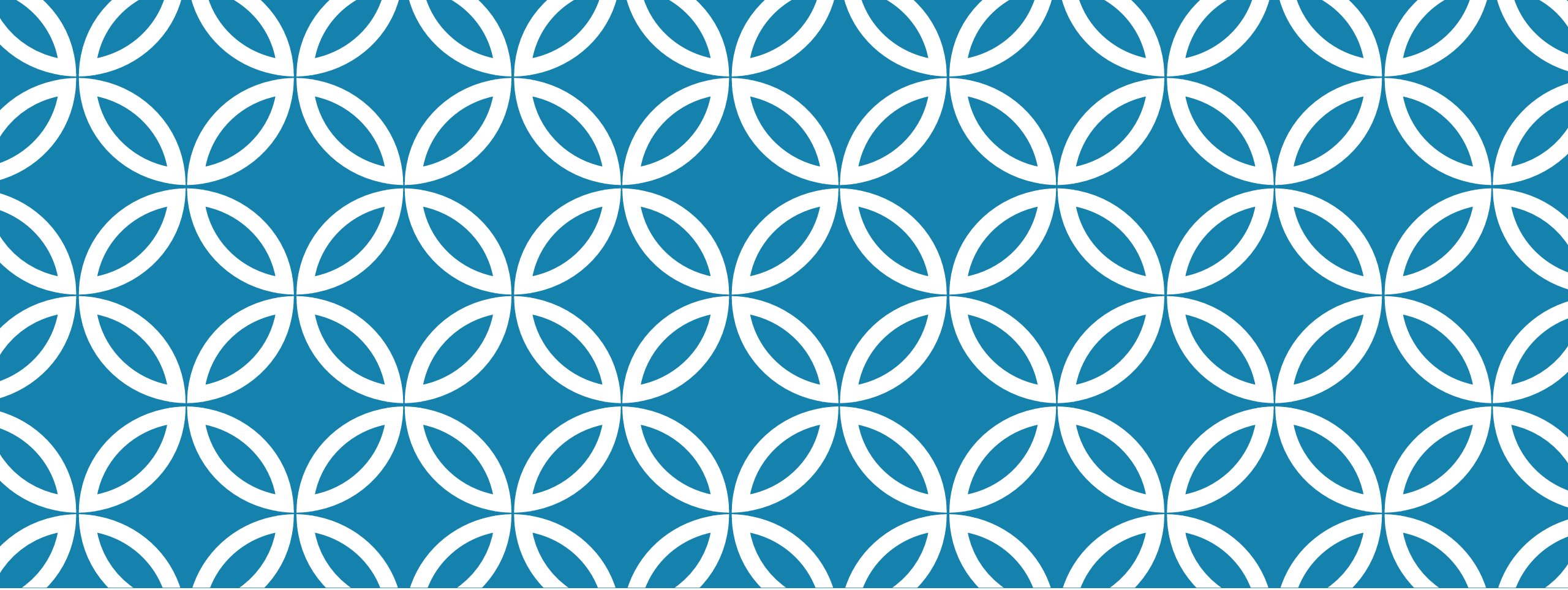
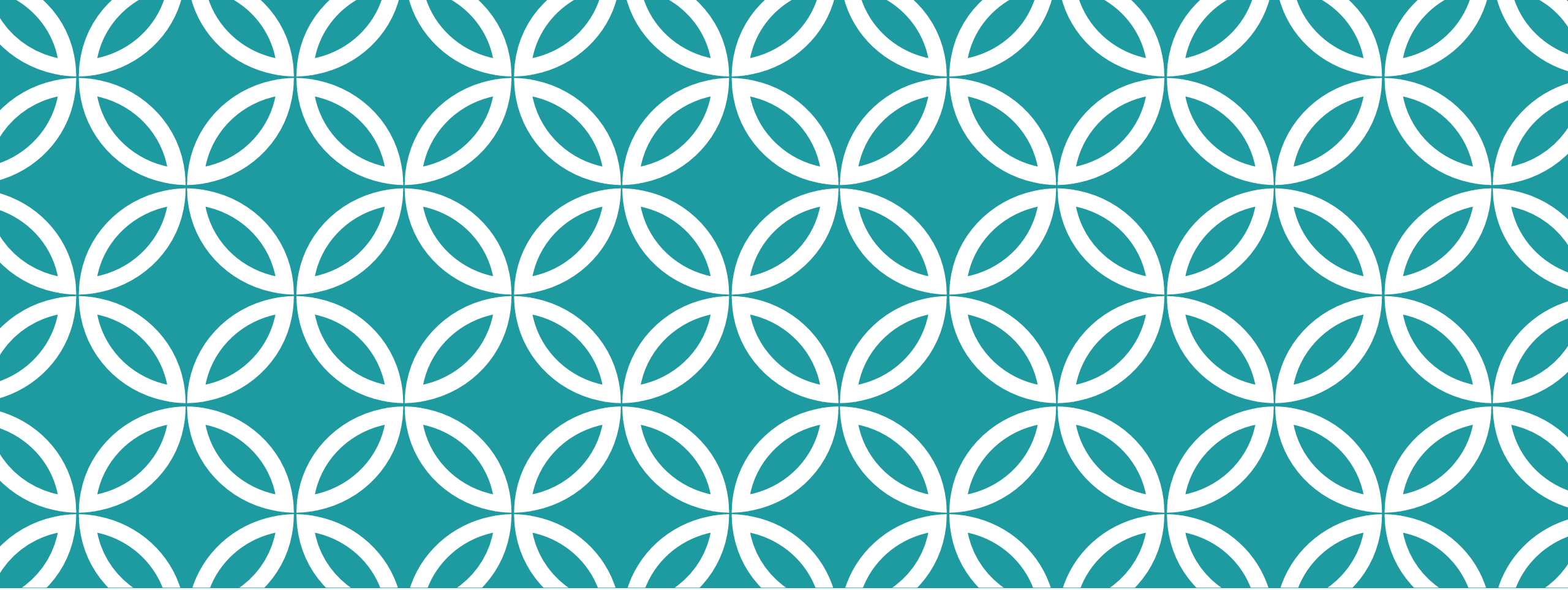


OOP – R6 SYSTEM

dr Maria Kubara, WNE UW



R6

Encapsulated OOP

R6

- encapsulated OOP – methods belong to objects, not generics – object$method()
- R6 objects are **mutable** – can be modified in place
- It is similar to the base OOP system **reference class** (RC) – however it is more efficient, as R6 is based on S3, and RC is based on S4
- Because it is based on S3, we can use S3 generics to work with R6 objects
- Curiosity about the name: we have S3, S4 which are following iterations of OOP stemming from S language – introduced first in 3rd version of S (language prior to R). RC reference class is a base OOP system in R, which supports encapsulation. R5 was introduced to follow S3 and S4, this time with encapsulation like in RC. However, R5 Github project was abandoned after little development. R6 is the project which finally develops the ideas that were to be developed in R5.

CLASS DEFINITION IN R6

R6 uses the encapsulated paradigm – methods are defined with classes.

Classes in R6 are created with R6::R6Class() function. This is the only function we will use from R6 package.

First argument – classname

Second argument – public → list of fields and methods (functions) belonging to the object

New object is created with className$new()

New class is created here

All fields need to have their default values specified

```
client6 <- R6Class("client6",
                   public = list(
                     fname = NULL,
                     lname = NULL,
                     age = 18,
                     gender = NA,
                     married = NA
                   ))

# Creating of new R6 objects with new:
k6 <- client6$new()
k6 # all default fields
```

New objects can be created using className$new()

If method initialize is not defined, no modification of fields is possible (default values are used).

CONSTRUCTOR IN R6

Methods in R6 are defined as other fields of the class.

To create proper constructor function which allows for field modification one needs to specify a public field named **initialize**

This method will be called with className$new()

Methods can access public fields of the current object via self$ and private elements via private$

All R6 methods by default return self invisibly (without printing in the console). This will be used for method chaining

```
client6 <- R6Class("client6",
                   public = list(
                     fname = NULL,
                     lname = NULL,
                     age = 18,
                     gender = NA,
                     married = NA,

                     initialize = function(fname, lname, age,
                                           gender, married){

                       self$fname = fname
                       self$lname = lname
                       self$age = age
                       self$gender = gender
                       self$married = married

                       invisible(self)
                       # this statement is a default one for R6
                       # invisible returning of the object
                       # we work on
                     }
                   ))

k6 <- client6$new(fname = "John",
                  lname = "Smith",
                  age = 35,
                  gender = "M",
                  married = TRUE)
```

New public field initialize – a function

Fields of the current objects can be referred to with self$ (for public fields) and private$ (for private fields)

After initialize is set the constructor can fill in the fields

VALIDITY CHECKING IN R6

Validity in R6 is like in S3 – we need to DIY

Supplementing initialize with defensive programming assertions makes the function prepared for possible user mistakes.

```
initialize = function(fname, lname, age, gender, married){
  # first some validity checking
  stopifnot(is.character(fname), is.character(lname))
  stopifnot(is.numeric(age), round(age,0)==age) #two conditions
  stopifnot(gender %in% c("M", "F"))
  stopifnot(is.logical(married))

  self$fname = fname
  self$lname = lname
  self$age = age
  self$gender = gender
  self$married = married

  invisible(self)
}
```

METHODS IN R6

R6 utilizes a nice* feature in R: function definitions are actually stored within variables

*"nice" depends on your previous programming experiences (it's more like in JavaScript than in Java)

```
> myVar <- "variable content"
> myFun <- function(x) print("function content")
>
> # We can check the content of both "variables"
> myVar
[1] "variable content"
> myFun # "variable" named myFun stores a function definition
function(x) print("function content")
> myFun() # function is run when () are used
[1] "function content"
```

Because functions are stored within variables…
we can add them as fields within lists and call them like other list elements by objectName$functionInside
For triggering action, we will close the call with brackets objectName$functionInside()

DEFINING R6 METHODS

Methods in R6 are defined as other fields of the class.

Just like with constructor – remember about accessing public fields of the current object via self$ and private elements via private$

After running makeOlder the object will be **mutated** – fields **will be updated** using the logic of the function (no need to use assignment symbol to apply changes). *R6 objects are mutable.*

```
client6 <- R6Class("client6",
                   public = list(
  fname = NULL, lname = NULL,
  age = 18, gender = NA, married = NA,
  initialize = function(fname,
                        lname, age,
                        gender, married){
    self$fname = fname
    self$lname = lname
    self$age = age
    self$gender = gender
    self$married = married

    invisible(self)
  }, #comma because we are adding another field

  # new element in the class - makeOlder function
  # which by default makes the client
  # older by one year
  makeOlder = function(olderBy=1){
    self$age <- self$age + olderBy
    invisible(self)
  }
))
```

makeOlder is a field within a class – this time it is a function (method)

```
k6$makeOlder()
```

This code will make k6$age field change by one – the default value in makeOlder function. Change will be saved automatically to the object field – **mutability**!

ADDING ELEMENTS AFTER CLASS CONSTRUCTION

Method $set() allows to add fields to the class after it's defined

As the class definition grows it may be difficult to keep everything with the first R6Class() call.

$set() allows to add new fields to the class – both methods and standard fields (with providing their defaults)

We need to specify the visibility (public/private), name (in quotes) and definition/default value.

```
client6$set("public", "ID", 1)
# adding an exemplary field with its default

client6$set("public", "makeMarried", function(){
  # see that the field name is within quotes
  self$married <- TRUE
  invisible(self)
})
```

Running this will change the class structure to include these fields. They will be available in the newly created objects.

Important: if you have created objects before the change, you need to recreate them to be able to use the new fields.

INHERITANCE IN R6

Inheritance is defined with the inherit field of the R6Class() function

All fields and methods will be inherited from the parent class (superclass).

→ if they are not overridden

We can utilize parent's method implementation for the child's version using super$functionName() – this works like NextMethod from S3 – runs the implementation of the parent's method and allows us to add any necessary modifications.

```
client6Premium <- R6Class("client6Premium",
                          inherit = client6,
                          public = list(
                            sayHello = function() {
                              super$sayHello()
                              # We can call the superclass (parent class)
                              # implementation here with super$...
                              # like with NextMethod() from S3.

                              # Then our printing behaviour can be modified
                              cat("I am a premium client", "\n")
                            }
                          )
```

New line after running parent's class method

RESTRICTING ACCESS IN R6

We have two levels of access in R6: public and private.

Public fields are available for the user in the console – one can call the field and even modify it:

```
object$age

object$age <- -20 #negative!
```

This allows for unwanted changes in the objects, when user applies modification which wouldn't have been allowed by the validity checking in the constructor function.

Private fields are only available from the class – we can get to them from the class definition (or from its methods).

If a field is private user cannot access it freely. We can allow for its changes via special methods – this allows to create appropriate checkers.

E.g., set age to private field and build two public methods: showAge() and changeAge() which will be safekeepers.

We can define private methods as well – these will be usually the methods which are reused by public methods of the function

PRIVATE FIELD - AGE

Private fields are defined within private argument in R6Class()

Modification or reading of private fields in the console can be guarded by purposefully designed methods – like showAge() and changeAge().

Inside the modification method we can define any necessary validity checkers to ensure proper structure of these elements.

```
client6 <- R6Class("client6",
                   private = list(age = 18),

                   public = list(
                     fname = NULL, # ...
                     initialize = function(fname, age){
                       # validity ...

                       self$fname = fname
                       private$age = age

                       invisible(self)
                     },

                     showAge = function(){
                       return(private$age) #visibly return age
                     },

                     changeAge = function(newAge){
                       stopifnot(is.numeric(newAge)) # numeric
                       stopifnot(newAge>0) # positive
                       stopifnot(newAge==round(newAge,0)) # no decimal

                       private$age <- newAge

                       invisible(self)

                     }
                   )

)
```

Age is a private field – k6$age won't work

Notice the change between self$ and private$

Private method for showing age

Special method changeAge() with validity checking of any age updates

ACTIVE FIELDS

Active fields look like simple fields from the outside, but are actually defined with functions

Another way to create fields which are protected from unnecessary modification is via the active fields.

We specify an active field in private elements of the class, and then we supply a look-alike function which defines the way the object is used (in active parameter).

```
client6 <- R6Class("client6",
                   private = list(age = 18,
                                  .fname = NULL,
                                  .lname = NULL),

  active = list(
    fname = function(value){
      if(missing(value)){
        private$.fname
      } else {
        stopifnot(is.character(value))
        private$.fname <- value
        invisible(self)
      }
    },
    lname = function(value){
      if(missing(value)){
        private$.lname
      } else {
        stop("Last name is for read only!")
      }
    }
  ),
  public = list(
    gender = NA,
    initialize = function(fname, lname, age, gender){
      # validity...
      private$.fname = fname #active
      private$.lname = lname
      private$age = age # private
      self$gender = gender # public
```

Active fields defined within private elements – no free access

Names are marked with a dot

Within active parameter we define the function which works when object$fname is called

if(missing(value)) defines the behavior for simple call: object$fname → fname will be printed

else statement in these functions navigates the behavior for object$name <- "some value" here for lname changes are not allowed, and for fname changes are allowed for character inputs

In constructor we also use the dots to access active fields

ACTIVE FIELDS - RECAP

```
active = list(
  fname = function(value){
    if(missing(value)){
      private$.fname
    } else {
      stopifnot(is.character(value))
      private$.fname <- value
      invisible(self)
    }
  },
```

Active field named .fname

Trial of access from the console:

```
object$fname
```

value is missing() (statement if(missing(value)) returns TRUE), define the behavior for retrieving the field value via private$.fname

```
object$fname <- "replacement"
```

Trying to put a value in the active field (here "replacement"). Else statement defines which actions are allowed

USING S3 GENERICS

We can use S3 generics with R6 classes

Methods in R6 can be used in two ways – in encapsulated way object$method() or in the typical R way method(object), if we create it in the S3 way – through generic and its additional method

This is done because all R6 classes inherit from general R6 class for which some of these methods are already implemented – like print.R6 – so we can see class structure printout in the console

Our class client6 inherits from class R6 implemented in R6:: package

We have print.R6 method defined in the R6:: package so print() works on all R6 objects – shows their structure

```
> class(k6)
[1] "client6" "R6"
> print(k6)
<client6>
  Public:
    age: 41
    clone: function (deep = FALSE)
    fname: John
    gender: M
    initialize: function (fname, lname, age, gender, married
    lname: Smith
    makeOlder: function (olderBy = 1)
    married: TRUE
> print.client6 <- function(x){
+   cat("This is client6 printing function", "\n")
+   cat("Hi, my name is", x$fname, x$lname, "\n")
+   cat("I am", x$age, "years old.", "\n")
+
+ }
>
> print(k6) # wow, this actually worked!
This is client6 printing function
Hi, my name is John Smith
I am 41 years old.
```

Sneaky way to put a method for client6 to an existing S3 generic